Java Track — Day 2 Assignment

Problem Solving Techniques and Data Structures — II

Solutions and sample console output

Mavericks Segue 2026 | 30 Oct 2026

Prepared in the same task-by-task format as the Day 1 solutions.

How to use this document

This document follows the three sets and task order in the supplied Day 2 assignment. Each task includes a Java example, illustrative console output, and a short explanation of the chosen data structure or algorithm. The snippets are designed to be copied into a Java class and run individually; helper classes are included where needed.

Note: The outputs below are expected sample outputs, not screenshots from an actual Java run. Timing measurements vary by computer, JVM, and input distribution. For a submission requiring execution evidence, run the code locally and replace the sample output with your own console screenshots.

Complexity reference

Technique	Typical time	Space	Use
BST search/insert	O(h); average O(log n), worst O(n)	O(h) recursive	Ordered lookup
HashMap lookup	Expected O(1)	O(n) total	Key-based lookup
Graph BFS / DFS	O(V + E)	O(V)	Reachability / traversal
Sliding window	O(n)	O(1)	Fixed-size contiguous range
Two pointers	O(n) after sorting	O(1) extra	Pairs / merge / palindrome

Set 1 — E-Commerce Catalogue and Traffic Analysis

Business scenario: An e-commerce platform needs faster category browsing, instant SKU look-ups, and a way to size infrastructure for peak traffic.

Task 1. Binary Search Tree of product categories

Java code

```
class Node {
    String key; Node left, right;
    Node(String key) { this.key = key; }
}
static Node insert(Node root, String key) {
    if (root == null) return new Node(key);
    int c = key.compareTo(root.key);
    if (c < 0) root.left = insert(root.left, key);
    else if (c > 0) root.right = insert(root.right, key);
    return root;
}
static boolean search(Node root, String key) {
    if (root == null) return false;
    if (root.key.equals(key)) return true;
    return key.compareTo(root.key) < 0
        ? search(root.left, key) : search(root.right, key);
}
static void in(Node n) { if(n!=null){in(n.left);System.out.print(n.key+" ");in(n.right);} }
static void pre(Node n) { if(n!=null){System.out.print(n.key+" ");pre(n.left);pre(n.right);} }
static void post(Node n) { if(n!=null){post(n.left);post(n.right);System.out.print(n.key+" ");} }

public static void main(String[] args) {
    String[] a = {"Electronics","Books","Clothing","Laptops","Mobiles",
      "Fiction","Nonfiction","Men","Women","Shoes","Home","Kitchen",
      "Furniture","Sports","Toys"};
    Node root = null;
    for (String s : a) root = insert(root, s);
    System.out.print("Inorder: "); in(root);
    System.out.print("\nPreorder: "); pre(root);
    System.out.print("\nPostorder: "); post(root);
    System.out.println("\nSearch Mobiles: " + search(root,"Mobiles"));
}
```

Sample output

Command Prompt | Java output

```
Inorder: Books Clothing Electronics Fiction Furniture Home Kitchen Laptops Men Mobiles Nonfiction Shoes Sports Toys Wo
Preorder: Electronics Books Clothing Fiction Furniture Home Kitchen Laptops Men Mobiles Nonfiction Shoes Sports Toys W
Postorder: Clothing Books Furniture Kitchen Home Fiction Men Nonfiction Mobiles Laptops Shoes Women Toys Sports Electr
Search Mobiles: true
```

Illustrative sample — run locally to verify.

Explanation: Inorder traversal prints keys in sorted lexicographical order for a BST. Search and insertion take O(h), where h is tree height; a balanced tree is O(log n), while a skewed tree can be O(n). The sample inserts 15 unique category names.

Task 2. Hash table with chaining and built-in HashMap

Java code

```
import java.util.*;

static class ChainedHash {
    List<String>[] buckets;
    ChainedHash(int size) {
        buckets = new ArrayList[size];
        for (int i=0;i<size;i++) buckets[i]=new ArrayList<>();
    }
    int index(String key) { return Math.floorMod(key.hashCode(), buckets.length); }
    void put(String key) { buckets[index(key)].add(key); }
    boolean contains(String key) { return buckets[index(key)].contains(key); }
}
public static void main(String[] args) {
    ChainedHash table = new ChainedHash(5);
    table.put("SKU-100"); table.put("SKU-105");
    System.out.println("Collision bucket: " +
        table.index("SKU-100") + " and " + table.index("SKU-105"));
    System.out.println("Chaining lookup SKU-105: " + table.contains("SKU-105"));

    Map<String,Integer> map = new HashMap<>();
    for (int i=0;i<100_000;i++) map.put("SKU-"+i,i);
    long start=System.nanoTime();
    for (int i=0;i<100_000;i++) map.get("SKU-"+i);
    long elapsed=System.nanoTime()-start;
    System.out.println("HashMap records: "+map.size());
    System.out.println("100000 lookups ns: "+elapsed);
}
```

Sample output

Command Prompt | Java output

```
Collision bucket: 1 and 1
Chaining lookup SKU-105: true
HashMap records: 100000
100000 lookups ns: [machine-dependent]
```

Illustrative sample — run locally to verify.

Explanation: Separate chaining stores colliding keys in the same bucket, usually as a list. The example deliberately maps SKU-100 and SKU-105 to the same bucket with five buckets. HashMap handles collisions internally. Expected lookup is O(1), but can degrade under heavy collisions. Run the benchmark several times and compare on the same machine; timing is not a fixed expected value.

Task 3. Microservice dependencies as a graph

Java code

```
import java.util.*;
public static void main(String[] args) {
    Map<String,List<String>> g = new LinkedHashMap<>();
    g.put("cart", List.of("pricing"));
    g.put("pricing", List.of("inventory"));
    g.put("inventory", List.of("payment"));
    g.put("payment", List.of());
    // Kahn's BFS topological ordering
    Map<String,Integer> indegree = new HashMap<>();
    for(String v:g.keySet()) indegree.put(v,0);
    for(String v:g.keySet()) for(String w:g.get(v))
        indegree.put(w,indegree.get(w)+1);
```

```
    Queue<String> q=new ArrayDeque<>();
    for(String v:g.keySet()) if(indegree.get(v)==0) q.add(v);
    List<String> order=new ArrayList<>();
    while(!q.isEmpty()){
        String v=q.remove(); order.add(v);
        for(String w:g.get(v)){
            indegree.put(w,indegree.get(w)-1);
            if(indegree.get(w)==0) q.add(w);
        }
    }
    System.out.println("Deployment order: "+order);
    System.out.println("Circular dependency: "+(order.size()!=g.size()));
}
```

Sample output

Command Prompt | Java output
`Deployment order: [cart, pricing, inventory, payment]` `Circular dependency: false`
Illustrative sample — run locally to verify.

Explanation: The directed edge A → B means A must be deployed before B in this example. Kahn's BFS-based topological sort produces a valid order. If fewer than all vertices are processed, a directed cycle exists. Time and space are O(V + E). DFS can also be used for cycle detection.

Task 4. Busiest continuous 3-hour window

Java code

```
public static void main(String[] args) {
    int[] rpm = {120,150,180,90,210,240,200,160,130,110,100,90,
                 80,100,120,140,160,180,200,220,240,260,230,190};
    int k=3, sum=0;
    for(int i=0;i<k;i++) sum+=rpm[i];
    int best=sum, bestStart=0;
    for(int i=k;i<rpm.length;i++){
        sum += rpm[i]-rpm[i-k];
        if(sum>best){best=sum;bestStart=i-k+1;}
    }
    System.out.println("Busiest 3-hour window: hours "+
        bestStart+"-"+(bestStart+2));
    System.out.println("Requests in window: "+best);
    System.out.println("Average RPM: "+(best/3.0));
}
```

Sample output

Command Prompt | Java output

`Busiest 3-hour window: hours 19-21`
`Requests in window: 720`
`Average RPM: 240.0`

Illustrative sample — run locally to verify.

Explanation: A fixed-size sliding window computes the first three-hour sum, then subtracts the outgoing hour and adds the incoming hour. This avoids recalculating every window from scratch. The algorithm runs in O(n) time and O(1) extra space. The sample array represents 24 hourly request counts.

Task 5. Two pointers to find gift-card pairs

Java code

```
import java.util.*;
public static void main(String[] args) {
    int[] values={10,20,30,40,50,60,70};
    int target=90, left=0, right=values.length-1;
    while(left<right){
        int sum=values[left]+values[right];
        if(sum==target){
            System.out.println(values[left]+" + "+values[right]);
            left++; right--;
        } else if(sum<target) left++;
        else right--;
```

```
    }
}
```

Sample output

Command Prompt | Java output

`20 + 70`
`30 + 60`
`40 + 50`

Illustrative sample — run locally to verify.

Explanation: Because the order values are sorted, two pointers can move inward based on whether the current sum is below or above the target. The scan takes O(n) time and O(1) extra space. If the input is not already sorted, sorting first costs O(n log n).

Set 2 — Insurance Claims Processing

Business scenario: An insurer routes claims through approval stages, detects duplicate submissions, and monitors claim spikes after a regional event.

Task 1. BST of claim IDs and traversals

Java code

```
class Node {
    int id; Node left,right;
    Node(int id){this.id=id;}
}
static Node add(Node n,int x){
    if(n==null)return new Node(x);
    if(x<n.id)n.left=add(n.left,x);
    else if(x>n.id)n.right=add(n.right,x);
    return n;
}
static boolean has(Node n,int x){
    if(n==null)return false;
    if(n.id==x)return true;
    return x<n.id?has(n.left,x):has(n.right,x);
}
static void inorder(Node n){
    if(n!=null){inorder(n.left);System.out.print(n.id+" ");inorder(n.right);}
}
static void preorder(Node n){
    if(n!=null){System.out.print(n.id+" ");preorder(n.left);preorder(n.right);}
}
static void postorder(Node n){
    if(n!=null){postorder(n.left);postorder(n.right);System.out.print(n.id+" ");}
}
public static void main(String[] args){
    int[] ids={50,30,70,20,40,60,80};
    Node root=null; for(int x:ids)root=add(root,x);
    System.out.print("Inorder: ");inorder(root);
    System.out.print("\nPreorder: ");preorder(root);
    System.out.print("\nPostorder: ");postorder(root);
    System.out.println("\nClaim 60 found: "+has(root,60));
}
```

Sample output

Command Prompt | Java output

```
Inorder: 20 30 40 50 60 70 80
Preorder: 50 30 20 40 70 60 80
Postorder: 20 40 30 60 80 70 50
Claim 60 found: true
```

Illustrative sample — run locally to verify.

Explanation: Inorder traversal gives claim IDs in ascending sorted order because this is a BST. Preorder visits root-left-right; postorder visits left-right-root. Insert and search are O(h), average O(log n) for a reasonably balanced tree and worst-case O(n).

Task 2. Hashing to detect duplicate or near-duplicate claims

Java code

```
import java.util.*;
static String normalize(String s){
    return s.replaceAll("\\s+","").toUpperCase(Locale.ROOT);
}
public static void main(String[] args){
    String[] policy={" ab 120 ","AB120","xy-450"};
    String[] date={"2026-10-01","2026-10-01","2026-10-02"};
    Set<String> seen=new HashSet<>();
    for(int i=0;i<policy.length;i++){
        String key=normalize(policy[i])+"|"+date[i];
        if(!seen.add(key)) System.out.println("DUPLICATE CLAIM: "+key);
        else System.out.println("ACCEPTED: "+key);
    }
}
```

Sample output

Command Prompt | Java output

```
ACCEPTED: AB120|2026-10-01
DUPLICATE CLAIM: AB120|2026-10-01
ACCEPTED: XY-450|2026-10-02
```

Illustrative sample — run locally to verify.

Explanation: Normalize policy numbers by removing whitespace and converting to a consistent case before building the key with the incident date. HashSet detects repeated normalized keys with expected O(1) membership checks. Hash collisions do not automatically mean duplicate claims: the set checks key equality after hashing. Total expected time and space are O(n).

Task 3. Approval stages graph: BFS shortest path and DFS cycle check

Java code

```
import java.util.*;
public static void main(String[] args){
    Map<String,List<String>> g=new LinkedHashMap<>();
    g.put("registration",List.of("assessment"));
    g.put("assessment",List.of("surveyor","settlement"));
    g.put("surveyor",List.of("settlement"));
    g.put("settlement",List.of());
    String start="registration", target="settlement";
    Queue<String> q=new ArrayDeque<>();
    Map<String,String> parent=new HashMap<>();
    q.add(start); parent.put(start,null);
    while(!q.isEmpty()){
        String v=q.remove();
        if(v.equals(target))break;
        for(String w:g.get(v)) if(!parent.containsKey(w)){
            parent.put(w,v);q.add(w);
        }
    }
    List<String> path=new ArrayList<>();
    if(parent.containsKey(target)){
        for(String v=target;v!=null;v=parent.get(v))path.add(v);
        Collections.reverse(path);
    }
    System.out.println("Shortest path: "+path);
    // DFS cycle detection with recursion-stack states
    Set<String> visiting=new HashSet<>(), done=new HashSet<>();
    System.out.println("Cycle detected: "+cycle("registration",g,visiting,done));
}
static boolean cycle(String v,Map<String,List<String>> g,
                     Set<String> visiting,Set<String> done){
    if(visiting.contains(v))return true;
    if(done.contains(v))return false;
    visiting.add(v);
    for(String w:g.get(v))if(cycle(w,g,visiting,done))return true;
    visiting.remove(v);done.add(v);return false;
}
```

Sample output

Command Prompt | Java output

```
Shortest path: [registration, assessment, settlement]
Cycle detected: false
```

Illustrative sample — run locally to verify.

Explanation: BFS finds the path with the fewest edges in an unweighted directed graph. DFS uses a recursion-stack set to detect a back edge and therefore a directed cycle. Both traversals take O(V + E) time and O(V) auxiliary space. The example includes a direct assessment-to-settlement route and an alternate surveyor route.

Task 4. Longest consecutive run of days with claims above 100

Java code

```
public static void main(String[] args){
    int[] claims={80,120,135,150,90,110,125,140,160,70,105,115};
    int best=0, current=0, bestEnd=-1;
```

```
    for(int i=0;i<claims.length;i++){
        if(claims[i]>100){
            current++;
            if(current>best){best=current;bestEnd=i;}
        } else current=0;
    }
    int start=bestEnd-best+1;
    System.out.println("Longest run length: "+best);
    System.out.println("Day indexes: "+start+" to "+bestEnd);
}
```

Sample output

Command Prompt | Java output
`Longest run length: 4` `Day indexes: 5 to 8`
Illustrative sample — run locally to verify.

Explanation: The condition is claims greater than 100. A running counter increases while the condition remains true and resets to zero when it fails. This finds the longest consecutive run in O(n) time and O(1) extra space. This is a single-pass consecutive-run scan; it is not a fixed-width window.

Task 5. Merge two sorted regional claim lists using two pointers

Java code

```
import java.util.*;
public static void main(String[] args){
    int[] north={101,104,109,115};
    int[] south={102,104,110,120};
    int i=0,j=0,k=0;
    int[] merged=new int[north.length+south.length];
    while(i<north.length && j<south.length){
        if(north[i]<=south[j])merged[k++]=north[i++];
        else merged[k++]=south[j++];
    }
    while(i<north.length)merged[k++]=north[i++];
    while(j<south.length)merged[k++]=south[j++];
    System.out.println("Merged claims: "+Arrays.toString(merged));
}
```

Sample output

Command Prompt | Java output
`Merged claims: [101, 102, 104, 104, 109, 110, 115, 120]`
Illustrative sample — run locally to verify.

Explanation: Two pointers track the current unmerged element in each already-sorted list. The smaller element is copied to the result, then its pointer advances. Remaining elements are appended. The merge takes O(n + m) time and O(n + m) space for the output array, without sorting the combined list.

Set 3 — Logistics Network and Delivery Windows

Business scenario: A courier company optimises hub-to-hub routing and delivery-window planning across a national network.

Task 1. BST of hub codes, height, and sorted insertion

Java code

```
class Node {
    String code; Node left,right;
    Node(String code){this.code=code;}
}
static Node insert(Node n,String x){
    if(n==null)return new Node(x);
    int c=x.compareTo(n.code);
    if(c<0)n.left=insert(n.left,x);
    else if(c>0)n.right=insert(n.right,x);
    return n;
}
static boolean search(Node n,String x){
    if(n==null)return false;
    if(n.code.equals(x))return true;
    return x.compareTo(n.code)<0?search(n.left,x):search(n.right,x);
}
static int height(Node n){
    if(n==null)return 0;
    return 1+Math.max(height(n.left),height(n.right));
}
public static void main(String[] args){
    String[] codes={"CHN","BLR","CBE","HYD","DEL","MUM","PUN"};
    Node root=null;for(String s:codes)root=insert(root,s);
    System.out.println("Height (nodes): "+height(root));
    System.out.println("Search HYD: "+search(root,"HYD"));
    Node sorted=null;
    for(String s:new String[]{"A","B","C","D","E"})sorted=insert(sorted,s);
    System.out.println("Height after sorted insertion: "+height(sorted));
}
```

Sample output

Command Prompt | Java output

```
Height (nodes): 3
Search HYD: true
Height after sorted insertion: 5
```

Illustrative sample — run locally to verify.

Explanation: Height is measured here in number of nodes on the longest root-to-leaf path; an empty tree has height zero. The sample insertion order creates a balanced shape with height three. Sorted input repeatedly inserts to one side, producing a skewed tree and height n, so search and insertion can degrade to O(n).

Task 2. HashMap parcel lookup and duplicate scan at same hub

Java code

```
import java.util.*;
record Scan(String tracking,String hub){}
public static void main(String[] args){
    List<Scan> scans=List.of(
        new Scan("TRK101","CHN"),new Scan("TRK102","BLR"),
        new Scan("TRK101","CHN"),new Scan("TRK101","HYD"));
    Map<String,Set<String>> seen=new HashMap<>();
    for(Scan s:scans){
        Set<String> hubs=seen.computeIfAbsent(s.tracking(),k->new HashSet<>());
        if(!hubs.add(s.hub()))
            System.out.println("REPEATED SCAN: "+s.tracking()+" at "+s.hub());
    }
    System.out.println("Tracking lookup TRK102: "+seen.containsKey("TRK102"));
}
```

Sample output

Command Prompt | Java output

```
REPEATED SCAN: TRK101 at CHN
Tracking lookup TRK102: true
```

Illustrative sample — run locally to verify.

Explanation: The HashMap maps each tracking number to the set of hubs where it has been scanned. Adding a hub that is already in the set identifies a repeated scan at that same hub. Expected processing time is O(n) overall, with O(n) storage for tracking numbers and hub sets.

Task 3. Ten-hub network: BFS fewest hops and DFS reachability

Java code

```
import java.util.*;
public static void main(String[] args){
    Map<String,List<String>> g=new LinkedHashMap<>();
    g.put("CHN",List.of("BLR","HYD"));
    g.put("BLR",List.of("CBE","MYS"));
    g.put("HYD",List.of("PUN"));
    g.put("CBE",List.of("MUM"));
    g.put("MYS",List.of("CBE"));
    g.put("PUN",List.of("DEL"));
    g.put("MUM",List.of());
    g.put("DEL",List.of());
    g.put("KOL",List.of("DEL"));
    g.put("JAI",List.of("PUN"));
    // BFS shortest route from CHN to MUM
    Queue<String> q=new ArrayDeque<>();
    Map<String,String> parent=new HashMap<>();
    q.add("CHN");parent.put("CHN",null);
    while(!q.isEmpty()){
        String v=q.remove();
        for(String w:g.get(v))if(!parent.containsKey(w)){
            parent.put(w,v);q.add(w);
        }
    }
    List<String> path=new ArrayList<>();
    if(parent.containsKey("MUM")){
        for(String v="MUM";v!=null;v=parent.get(v))path.add(v);
        Collections.reverse(path);
    }
    Set<String> visited=new LinkedHashSet<>();
    dfs("CHN",g,visited);
    System.out.println("Fewest-hop route: "+path);
    System.out.println("Reachable from CHN: "+visited);
}
static void dfs(String v,Map<String,List<String>> g,Set<String> seen){
    if(!seen.add(v))return;
    for(String w:g.get(v))dfs(w,g,seen);
}
```

Sample output

Command Prompt | Java output

```
Fewest-hop route: [CHN, BLR, CBE, MUM]
Reachable from CHN: [CHN, BLR, CBE, MUM, MYS, HYD, PUN, DEL]
```

Illustrative sample — run locally to verify.

Explanation: The adjacency list stores each hub and its direct outgoing routes. BFS finds the route with the fewest hops in an unweighted network. DFS lists all hubs reachable from the central depot CHN; KOL and JAI are not reachable from CHN in this directed example. Both algorithms run in O(V + E) time.

Task 4. Four-hour window with highest average parcel count

Java code

```
public static void main(String[] args){
    int[] parcels={20,25,30,35,40,45,50,55,60,65,70,75};
    int k=4,sum=0;
    for(int i=0;i<k;i++)sum+=parcels[i];
    int best=sum,bestStart=0;
    for(int i=k;i<parcels.length;i++){
        sum+=parcels[i]-parcels[i-k];
        if(sum>best){best=sum;bestStart=i-k+1;}
```

```
    }
    System.out.println("Best 4-hour window: "+bestStart+"-"+(bestStart+3));
    System.out.println("Total parcels: "+best);
    System.out.println("Average per hour: "+(best/(double)k));
}
```

Sample output

Command Prompt | Java output

`Best 4-hour window: 8-11`
`Total parcels: 270`
`Average per hour: 67.5`

Illustrative sample — run locally to verify.

Explanation: Since every candidate window contains four hours, maximising the sum also maximises the average. The sliding window updates the total in constant time per hour, giving O(n) time and O(1) extra space.

Task 5. Two-pointer palindrome check for route codes

Java code

```
public static void main(String[] args){
    String route="CHN-BLR / CBE / BLR-CHN";
    StringBuilder clean=new StringBuilder();
    for(char c:route.toUpperCase().toCharArray())
        if(Character.isLetterOrDigit(c))clean.append(c);
    int l=0,r=clean.length()-1;
    boolean palindrome=true;
    while(l<r){
        if(clean.charAt(l)!=clean.charAt(r)){palindrome=false;break;}
        l++;r--;
    }
    System.out.println("Normalised route: "+clean);
    System.out.println("Palindrome: "+palindrome);
}
```

Sample output

Command Prompt | Java output
`Normalised route: CHNBLRCBEBLRCHN` `Palindrome: false`
Illustrative sample — run locally to verify.

Explanation: The route is normalised by converting to uppercase and ignoring separators and spaces. Two pointers compare characters from the beginning and end, moving inward until a mismatch is found or the pointers meet. The check takes O(n) time and O(n) space for the normalised string; the comparison itself uses O(1) extra space.

Submission checklist

• Include Java source code for all tasks and organise each task in a separate class or clearly named method.

• Run every program and replace illustrative outputs with genuine console screenshots if your trainer requires execution evidence.

• For Set 1 Task 2, run the 100,000-SKU benchmark multiple times and report measured results from your computer.

• For graph tasks, include the adjacency list or a graph diagram and state whether edges are directed.

• For the sliding-window tasks, clearly state the window size and the selected starting and ending indexes or hours.

• Include the requested decision table for Set 1, complexity table for Set 2, and algorithm-choice explanation for Set 3.

Decision table — Set 1

Problem	Data structure / pattern	Reason
Product categories	Binary Search Tree	Supports ordered insert/search and sorted inorder traversal.
SKU lookup	Hash table / HashMap	Expected constant-time key lookup; chaining handles collisions.
Deployment dependencies	Directed graph + BFS/DFS	Models dependency edges, topological order and cycle detection.
Peak 3-hour traffic	Sliding window	Updates a fixed-size range in constant time per step.
Gift-card pairs	Two pointers	Uses sorted input to find target pairs in one linear scan.